

REST API for IoT Systems

Why Does an IoT Device Need a Server?

An IoT sensor collects data — but what happens next?

Temperature Sensor (ESP32)

"It's 72.4°F right now"



??? Where does this go?

Options:

- Store on device only → no remote visibility
- Send to a **server** → accessible anywhere, historical data, alerts

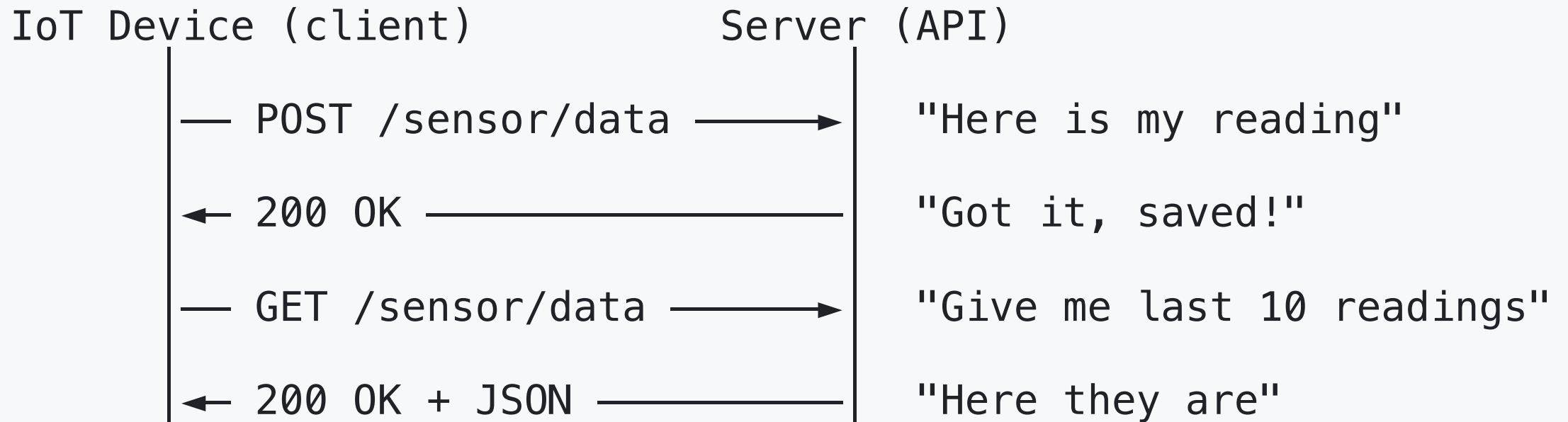
A **server** is a computer that waits for requests and serves responses — it gives your IoT data a home on the network.

What is a REST API?

REST = Representational State Transfer

API = Application Programming Interface

A REST API is a standard way for devices and applications to **communicate over HTTP**.



REST uses standard HTTP verbs: **GET, POST, PUT, DELETE**

- GET = read data
- POST = create new data
- PUT = update existing data
- DELETE = remove data

They are built into the HTTP protocol — no need for custom commands. Just use the right verb and path!

You can make your APIs such as `/api/sensor/data` for sensor readings, `/api/devices` for device registry, etc.

HTTP Request — Anatomy

there are multiple ways to make HTTP requests:

1. With a web browser (type URL, click buttons)
2. With command-line tools (curl)
3. With code (Python requests, JavaScript fetch)

Every HTTP request has these parts:

```
POST /api/sensor/temperature HTTP/1.1
Host: myiot-server.com
Content-Type: application/json
Authorization: Bearer eyJhbGci...
```

```
{
  "device_id": "esp32-001",
  "temperature": 72.4,
  "humidity": 45.2,
  "timestamp": "2024-11-01T10:30:00Z"
}
```


Part	Purpose
Method (POST)	What action to take (write)
Path (/api/sensor/temperature)	Which resource
Headers	Metadata (content type, auth token)
Body (JSON)	The data payload

HTTP Response — Anatomy

The server replies with a response:

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: Fri, 01 Nov 2024 10:30:01 GMT

{
  "status": "saved",
  "id": 4821,
  "timestamp": "2024-11-01T10:30:01Z"
}
```

HTTP Status Codes (Important!)

Code	Meaning	When
200 OK	Success	GET, PUT worked
201 Created	Resource created	POST worked
400 Bad Request	Your data is malformed	Wrong JSON
401 Unauthorized	Auth failed	Bad token
404 Not Found	Resource doesn't exist	Wrong path
500 Server Error	Server crashed	Bug in server

IoT Device Sending Data — ESP32 Example

```
// ESP32: POST sensor data to REST API
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

void sendSensorData(float temperature, float humidity) {
    HTTPClient http;
    http.begin("http://myiot-server.com/api/sensor/data");
    http.addHeader("Content-Type", "application/json");
    http.addHeader("Authorization", "Bearer my-secret-token");

    // Build JSON body
    StaticJsonDocument<200> doc;
    doc["device_id"] = "esp32-001";
    doc["temperature"] = temperature;
    doc["humidity"] = humidity;

    String body;
    serializeJson(doc, body);

    int code = http.POST(body);
    Serial.println("Response code: " + String(code));
    http.end();
}
```

Processing the Request from ESP32

We need to make server code that can handle the requests from the ESP32:

- Frontend: Web dashboard to display data
 - Written in JavaScript
 - For interacting with users through web browsers
- Backend: REST API to receive data and save to database
 - Written in any programming languages
 - Runs on a server, not visible to users

Frontend Responsibilities

The **frontend** is what the user sees and interacts with using web browsers.

- Users run this code in their browser to access the IoT data uploaded into the server.
- Then display the data in charts, tables, etc.

```
// JavaScript: Fetch latest sensor readings
async function fetchLatestReadings() {
  const response = await fetch("/api/sensor/data?limit=20", {
    headers: { Authorization: "Bearer " + userToken }
  });
  const data = await response.json();

  // Display on chart (Web Browser)
  updateChart(data.readings);
}
```

In most cases, we use JSON format for data exchange between frontend and backend:

Example response data

```
{
  "readings": [
    { "time": "10:30", "temp": 72.4, "humidity": 45.2 },
    { "time": "10:25", "temp": 71.8, "humidity": 44.9 }
  ]
}
```

Frontend never talks to the database directly — always through the API.

Backend Responsibilities

The **backend** (API server) is the business logic layer.

- The backend can process any requests from any device (ESP32, mobile app, web dashboard).
- It can validate the data, save it to a database, and send responses back to the clients.
- To display any information on the frontend, it should send HTML/JS/CSS files to the browser.
- To respond to API requests, it uses JSON data in most cases.

This is an example to respond to the POST request from ESP32 and save the data to a database:

```
@app.route('/api/sensor/data', methods=['POST'])
def receive_sensor_data():
    data = request.get_json()

    # Validate
    if 'temperature' not in data:
        return jsonify({"error": "missing temperature"}), 400

    # Save to database (pseudocode)
    record = {
        "device_id": data['device_id'],
        "temperature": data['temperature'],
        "humidity": data['humidity'],
        "received_at": datetime.utcnow().isoformat()
    }
    db.save(record) # save to database

    return jsonify({"status": "saved", "id": record['id']}), 201
```

This is written in Python using Flask, but you can use any language/framework (Node.js, Java, Go, etc.) to build the backend.

main.py

This is the main Python file to run the backend server:

```
from flask import Flask, request, jsonify
from datetime import datetime

app = Flask(__name__)

@app.route('/api/sensor/data', methods=['POST'])
def receive_sensor_data():
    ...

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=3000, debug=True)
```

Running the Backend Server (Python Example)

```
Client
  |
  v
Python Server (Uvicorn/Gunicorn)
  |
  v
Application Code
```

You need to run your Python code with a server that can handle multiple requests (like Uvicorn or Gunicorn) — not just the built-in Flask development server.

uvicorn vs Gunicorn

Development:

- `pip install uvicorn`
- `uvicorn --reload`
- Flask dev server

Production:

- `pip install gunicorn`
- `gunicorn -w 4 -b 0.0.0.0:8000 main:app` <- 4 workers,
bind to port 8000

Using the API

Now, any client (ESP32, web dashboard, mobile app) can send the POST requests to the API server to store sensor data in the Database.

```
curl -X POST http://localhost:3000/api/sensor/data \
-H "Content-Type: application/json" \
-d '{"device_id":"esp32-01","temperature":25.4,"humidity":60}'
```

This is an example using curl command to test the API endpoint from the command line.

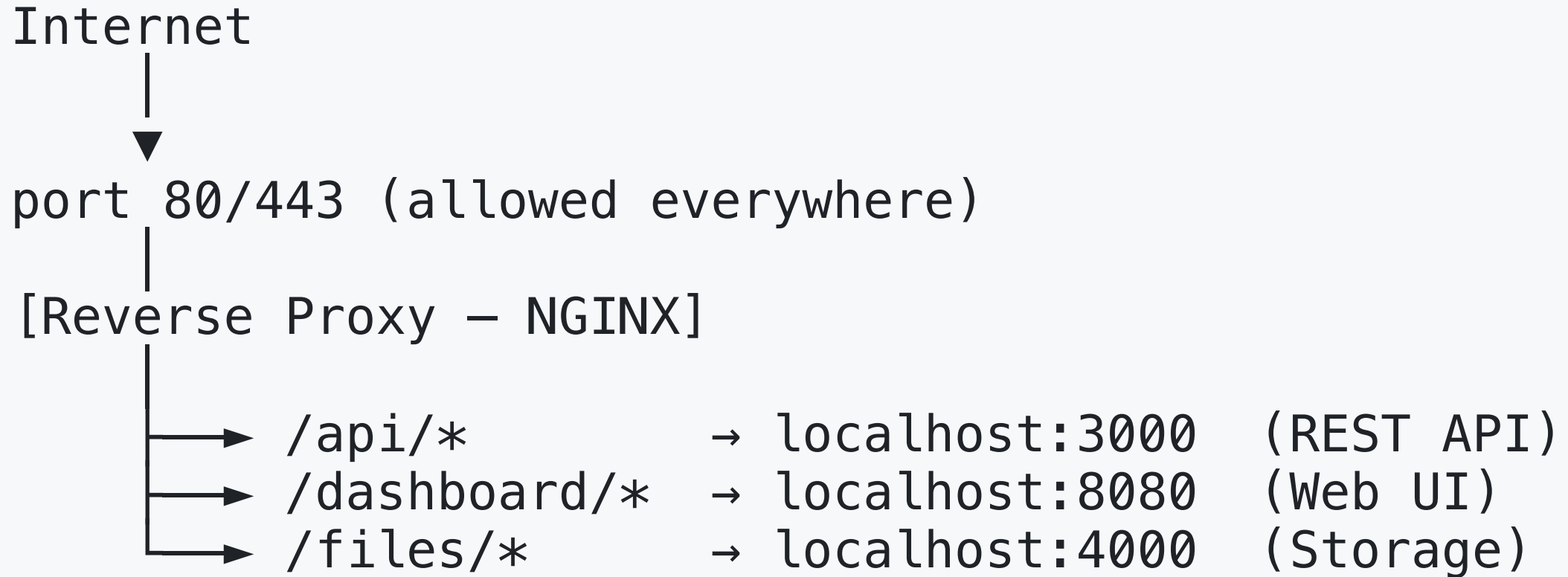
What is a Reverse Proxy?

Problem: You have multiple services on one server:

- REST API on port 3000
- MQTT dashboard on port 8080
- File storage on port 4000

Users can't access `myserver.com:3000` — firewalls block non-standard ports.

Solution: Reverse Proxy



A reverse proxy is a **traffic director** — it receives all requests and routes them to the right internal service.

NGINX as Reverse Proxy

NGINX (Engine-X) is the most popular reverse proxy for IoT backends.

- We need to configure NGINX to forward requests to the correct backend services based on the URL path.
- The configuration files are located in `/etc/nginx/sites-available/` on Linux servers.
- In this example, we forward `/api/*` to the REST API on port 3000, and serve the frontend dashboard from static files.


```
# /etc/nginx/sites-available/iot-server

server {
    listen 80;
    server_name myiot-server.com;

    # Forward /api requests to REST backend (port 3000)
    location /api/ {
        proxy_pass          http://localhost:3000;
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
    }

    # Serve frontend dashboard (static files)
    location / {
        root    /var/www/iot-dashboard;
        index   index.html;
        try_files $uri $uri/ /index.html;
    }
}
```

NGINX also handles **SSL/TLS (HTTPS)**, load balancing, caching, and rate limiting.

From Client to Your Code through NGINX

```
Client (Browser)
  | HTTPS :443 or HTTP :80
  v
Reverse Proxy (Nginx / Caddy)
  | HTTP (localhost)
  v
Python Server (Uvicorn / Gunicorn)
  | calls
  v
Your Application Code
```

Database Server for IoT Data

IoT generates **time-series data** — readings stamped with time.

device_id	temperature	humidity	timestamp	
esp32-001	72.4	45.2	2024-11-01	10:30:00
esp32-001	72.1	44.8	2024-11-01	10:35:00
esp32-002	68.0	60.1	2024-11-01	10:30:00

Two main database types for IoT:

Type	Example	Best For
Relational (SQL)	PostgreSQL, SQLite	Device registry, users, config
Time-Series	InfluxDB, TimescaleDB	Sensor readings over time

Relational Database (PostgreSQL) for IoT

Good for structured data: devices, users, configurations.

```
-- Create a table for IoT devices
CREATE TABLE devices (
    id          SERIAL PRIMARY KEY,
    device_id   VARCHAR(50) UNIQUE NOT NULL,
    name        VARCHAR(100),
    location    VARCHAR(200),
    registered  TIMESTAMP DEFAULT NOW()
);
```

```
-- Create a table for sensor readings
CREATE TABLE readings (
    id          BIGSERIAL PRIMARY KEY,
    device_id   VARCHAR(50) REFERENCES devices(device_id),
    temperature FLOAT,
    humidity    FLOAT,
    recorded_at TIMESTAMP DEFAULT NOW()
);

-- Query: average temperature last 24 hours per device
SELECT device_id, AVG(temperature) as avg_temp
FROM readings
WHERE recorded_at > NOW() - INTERVAL '24 hours'
GROUP BY device_id;
```

Time-Series Database (InfluxDB) for IoT

InfluxDB is purpose-built for sensor data — faster queries on time ranges.

```
# Python: Write sensor data to InfluxDB
from influxdb_client import InfluxDBClient, Point
from datetime import datetime

client = InfluxDBClient(url="http://localhost:8086", token="my-token")
write_api = client.write_api()

# Write a data point
point = (
    Point("sensor_reading")          # measurement name
    .tag("device_id", "esp32-001")   # indexed tag
    .tag("location", "room-A")
    .field("temperature", 72.4)      # value
    .field("humidity", 45.2)
    .time(datetime.utcnow())
)
write_api.write(bucket="iot-data", record=point)

# Query: average temp last 1 hour
query = '''
from(bucket:"iot-data")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "sensor_reading")
  |> mean()
'''
```

Complete IoT System Architecture

